# VitalHop

Multi-Hop LoRa Network for Offline Patient Vitals Monitoring Using Edge AI

## Core Idea — Simple Project Guide

VitalHop is an offline patient-monitoring system designed for rural and disaster-affected areas where Internet or cellular connectivity may be unavailable. A wearable device measures ECG, heart rate, SpO■ and temperature. An ESP32 processes the readings locally using TinyML and classifies the patient status as Normal, Warning or Emergency. Only important events are sent through a multi-hop LoRa network to a local PHC gateway and dashboard.

> **One-line idea:** Sense the patient → understand the condition at the edge → send important events through a reliable LoRa path → show them at the PHC without Internet.

## 1. What Problem Are We Solving?

In remote or disaster locations, Wi-Fi/GSM/Internet may not be reliable. Sending every raw sensor reading also wastes LoRa bandwidth and energy. A single communication path can fail, while emergency information must reach medical staff quickly.

## 2. What Is the Key Idea?

The key idea is **intelligent patient-event communication**. The ESP32 does not simply forward all sensor data. It first analyzes the patient's signals locally, creates a compact health-status event, gives that event a priority, and then the LoRa network delivers it using acknowledgements, retries, rerouting and buffering.

## 3. Complete System Flow

**Patient Sensors**
ECG + Heart Rate + SpO■ + Temperature
↓
**Patient ESP32**
Filtering + TinyML + Status Classification
↓
**LoRa Multi-Hop Network**
Relay A → Relay B → … → PHC Gateway
↓
**Local PHC Dashboard**
Patient status + alerts + event history + network status

## 4. Why Do We Need LoRa Hardware?

LoRa transceiver modules provide the physical long-range wireless communication. ESP32 firmware controls what is transmitted and how packets are handled. We do not need a separate new communication technology for the core project.

## 5. What Is Implemented in Code?

| Feature | What our code does |
|---|---|
| Packet format | Adds patient ID, node ID, sequence, status, priority, TTL and vital values. |
| ACK / Retry | Checks whether the next node received the packet and retries when needed. |

| Feature | What our code does |
|---|---|
| Priority | Emergency (P0) is transmitted before Warning (P1) and Routine (P2). |
| Dynamic rerouting | If a relay/path fails, the node attempts another available path. |
| Store-and-forward | If no route exists, important packets are buffered and sent when connectivity returns. |

## 6. The Three Traffic Priorities

| Priority | Meaning | Network action |
|---|---|---|
| P0 — Emergency | Critical patient event | Immediate transmission, higher retry priority, alternate route if nee |
| P1 — Warning | Potentially important event | Send before routine traffic |
| P2 — Routine | Normal/periodic information | Send when the channel is available |

## 7. What Makes VitalHop Unique?

• **Edge-event communication:** TinyML converts continuous sensor information into compact patient-status events before long-range transmission.

• **Priority-aware networking:** Emergency events receive preferential treatment over routine traffic.

• **Resilient multi-hop communication:** The network can attempt route recovery when a relay fails.

• **Store-and-forward:** Important events are retained during an outage and forwarded after a route becomes available.

• **Offline operation:** The field communication path does not depend on Internet or cellular connectivity.

## 8. Hardware Architecture

**Patient Node:** ESP32 + ECG sensor + MAX3010x-class sensor + temperature sensor + LoRa + battery.

**Relay Nodes:** ESP32 + LoRa + power supply. Relay nodes only forward packets; they do not need to run the health classifier.

**PHC Gateway:** ESP32 + LoRa. It can host a local web server and dashboard over local Wi-Fi. A Raspberry Pi is optional, not mandatory.

## 9. Important Communication Rule

We should not continuously stream high-rate raw ECG through every LoRa relay. Sensors can sample locally, while the network mainly carries compact event summaries and selected records. This keeps the communication load practical for LoRa.

## 10. Final Demonstration

The strongest demonstration is: (1) disconnect the Internet, (2) show live patient sensing and TinyML classification, (3) generate an Emergency event and show its priority transmission, (4) switch off a relay and show rerouting or buffering, (5) restore the route and show buffered delivery, and (6) display the received event on the local PHC dashboard.

## 11. Implementation Order

**1.** Sensor bring-up — test ECG, HR/SpO■ and temperature individually.

**2.** Wearable acquisition — combine sensors and create timestamped data.

**3.** TinyML — prepare data, train Normal/Warning/Emergency classifier and deploy it on ESP32.

**4.** LoRa point-to-point — establish reliable packet transmission with ACK/retry.

**5.** Multi-hop — add Relay A and Relay B.

**6.** Priority — implement P0/P1/P2 queues.

**7.** Rerouting — detect failed paths and select an alternate route.

**8.** Store-and-forward — buffer undelivered important packets and forward them after recovery.

**9.** PHC gateway — receive LoRa events and host the local dashboard.

**10.** Evaluation — measure ML performance, latency, delivery ratio, route recovery and buffer recovery.

## 12. What We Are NOT Claiming

VitalHop is an engineering/research prototype, not a clinically certified medical device. TinyML output should be described as experimental patient-status/risk classification, not as a medical diagnosis.

## 13. Core Research Claim

VitalHop aims to show that combining edge-based patient-event classification with priority-aware, resilient multi-hop LoRa communication can reduce unnecessary communication and improve delivery of important patient events during controlled offline and relay-failure conditions.

> **Project identity:** Sensors + ESP32 + TinyML + LoRa Mesh + Priority Routing + Dynamic Rerouting + Store-and-Forward + Local PHC Dashboard